

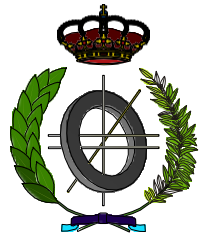


UNIVERSIDAD DE BURGOS  
ESCUELA POLITÉCNICA SUPERIOR  
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería  
Informática**

**SISTEMA DE GESTIÓN Y ASISTENTE  
DE DESPLIEGUES CI/CD EN  
ENTORNOS CLOUD (AWS)**



Presentado por David Tomé Arnaiz en la  
Universidad de Burgos — 5 de Junio de 2026

Tutor: José Manuel Aroca Fernández

## AGRADECIMIENTOS

En primer lugar, me gustaría agradecer a mi tutor José Manuel Aroca Fernández, la idea del proyecto, la orientación y disponibilidad para poder llevar a cabo este TFG, así como las sugerencias y correcciones.

También me gustaría agradecer a los profesores del Grado de Ingeniería Informática los conocimientos aprendidos a lo largo de la titulación, que han servido para desarrollar este proyecto.

## RESUMEN

El despliegue de aplicaciones en entornos Cloud es una tarea importante a la hora de desarrollar software, esto hace que los procesos sean cada vez más controlados y automatizados. La utilización de metodologías DevOps y de integración y despliegue continuo hace mejorar la calidad del software y reducir posibles fallos a la hora de ponerlo en producción.

Este TFG aborda este problema mediante el desarrollo de una aplicación web que sirve de asistente de despliegues, cuyo objetivo es acompañar al usuario en todo momento para conseguir la creación y configuración de aplicaciones orientadas a su despliegue automatizado en Cloud (AWS). La aplicación web guía al usuario a través de un flujo estructurados, permitiendo seleccionar un lenguaje de programación, herramienta de CI/CD, motores de bases de datos y configuraciones de entorno, generando las variables necesarias para conseguir su despliegue.

La solución planteada en el proyecto incluye conceptos relacionados con Cloud Computing, automatizaciones, calidad del código y gestión de configuración, ayudando al usuario a unas buenas prácticas en entornos profesionales. Hay que destacar que este proyecto tiene un carácter formativo, gracias al cual se ha ganado experiencia en el uso de servicios Cloud, especialmente en AWS, así como consolidar conocimientos técnicos aplicables al ámbito profesional.

El proyecto no solo sirve como objetivo académico, sino que también sirve como aprendizaje práctico y un escaparate de valor técnico para demostrar las competencias aprendidas durante el grado en relación con el desarrollo, despliegue y gestión de aplicaciones en la nube.

## Tabla de contenido

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>AGRADECIMIENTOS .....</b>                                    | <b>0</b>  |
| <b>RESUMEN .....</b>                                            | <b>0</b>  |
| <b>1. INTRODUCCIÓN.....</b>                                     | <b>3</b>  |
| 1.1 Contexto del Proyecto.....                                  | 3         |
| 1.2 Motivación.....                                             | 5         |
| 1.3 Justificación del trabajo .....                             | 5         |
| 1.4 Alcance del proyecto .....                                  | 5         |
| 1.5 Materiales y entregables .....                              | 5         |
| <b>2. OBJETIVOS DEL PROYECTO .....</b>                          | <b>7</b>  |
| 2.1 Objetivos funcionales .....                                 | 7         |
| 2.2 Objetivos técnicos .....                                    | 7         |
| 2.3 Objetivos académicos .....                                  | 9         |
| <b>3. CONCEPTOS TEÓRICOS .....</b>                              | <b>9</b>  |
| 3.1 DevOps como enfoque de cultura empresarial .....            | 9         |
| 3.2 Separación de entornos y gestión de configuración .....     | 10        |
| 3.3 Integración y despliegue continuo (CI/CD).....              | 10        |
| 3.4 Automatización de despliegues .....                         | 12        |
| 3.5 Asistente de despliegue de entornos Cloud .....             | 12        |
| <b>4. METODOLOGÍAS, HERRAMIENTAS Y TÉCNICAS. ....</b>           | <b>12</b> |
| 4.1 Metodología .....                                           | 13        |
| 4.1.1 Planificación del proyecto.....                           | 13        |
| 4.1.2 Uso de Trello para la gestión de tareas .....             | 13        |
| 4.2 Control de versiones .....                                  | 15        |
| 4.2.1 Uso de GitHub Actions.....                                | 16        |
| 4.2.2 Organización del repositorio .....                        | 16        |
| 4.2.3 Control de versiones en el proyecto .....                 | 17        |
| 4.3 Herramientas utilizadas.....                                | 17        |
| 4.3.1 Framework de desarrollo: IntelliJ IDEA .....              | 17        |
| 4.3.2 Contenedores .....                                        | 18        |
| 4.3.3 Bases de datos .....                                      | 19        |
| 4.3.4 Calidad del código .....                                  | 19        |
| <b>5. ASPECTOS RELEVANTES DEL DESARROLLO DEL PROYECTO .....</b> | <b>20</b> |

|            |                                                                     |                  |
|------------|---------------------------------------------------------------------|------------------|
| <b>5.1</b> | <b>Decisiones técnicas .....</b>                                    | <b>20</b>        |
| <b>5.2</b> | <b>Arquitectura del sistema.....</b>                                | <b>21</b>        |
| 5.2.1      | Arquitectura hexagonal.....                                         | 21               |
| 5.2.2      | Diagrama funcional de arquitectura .....                            | 22               |
| <b>5.3</b> | <b>Diseño del asistente de despliegues .....</b>                    | <b>23</b>        |
| 5.3.1      | Decisiones de diseño .....                                          | 25               |
| <b>5.4</b> | <b>Integración CI/CD .....</b>                                      | <b>25</b>        |
| 5.4.1      | Generación de configuración de despliegue .....                     | 26               |
| 5.4.2      | Gestión de variables sensibles .....                                | 26               |
| <b>5.5</b> | <b>Gestión de bases de datos .....</b>                              | <b>26</b>        |
| <b>5.6</b> | <b>Seguridad en las aplicaciones .....</b>                          | <b>27</b>        |
| <b>5.7</b> | <b>seguridad en el acceso a servicios de terceros .....</b>         | <b>28</b>        |
| <b>5.8</b> | <b>Manuales y tutoriales.....</b>                                   | <b>30</b>        |
| 5.8.1      | Wizards guiados .....                                               | 30               |
| 5.8.2      | Manuales de apoyo .....                                             | 30               |
| <b>6.</b>  | <b><i>TRABAJOS RELACIONADOS .....</i></b>                           | <b><i>31</i></b> |
| 6.1        | Proyectos y soluciones similares .....                              | 31               |
| 6.2        | Comparación con otros trabajos .....                                | 31               |
| 6.3        | Aportaciones del proyecto .....                                     | 32               |
| <b>7.</b>  | <b><i>USO DE INTELIGENCIA ARTIFICIAL EN EL DESARROLLO .....</i></b> | <b><i>32</i></b> |
| 7.1        | Contexto del uso de IA .....                                        | 32               |
| 7.2        | Ámbitos en los que se ha utilizado.....                             | 33               |
| 7.3        | Limitaciones y control del uso.....                                 | 33               |
| 7.4        | Responsabilidad del autor .....                                     | 33               |
| <b>8.</b>  | <b><i>RESULTADOS Y VALIDACIÓN .....</i></b>                         | <b><i>33</i></b> |
| 8.1        | Resultados obtenidos .....                                          | 33               |
| 8.2        | Validación funcional del sistema.....                               | 34               |
| 8.3        | Limitaciones actuales.....                                          | 34               |
| <b>9.</b>  | <b><i>CONCLUSIONES Y LÍNEAS DE TRABAJO FUTURAS .....</i></b>        | <b><i>35</i></b> |
| 9.1        | Conclusiones técnicas .....                                         | 35               |
| 9.2        | Conclusiones académicas .....                                       | 35               |
| 9.3        | Certificaciones técnicas relacionadas con el proyecto .....         | 35               |
|            | AWS Certified Cloud Practitioner .....                              | 36               |
|            | AWS Certified Solutions Architect- Associate .....                  | 36               |
| 9.4        | Trabajo futuro .....                                                | 37               |

|                                                |           |
|------------------------------------------------|-----------|
| <b>10. BIBLIOGRAFÍA.....</b>                   | <b>37</b> |
| <b>ANEXOS .....</b>                            | <b>39</b> |
| <b>Anexos A. Estructura del proyecto .....</b> | <b>39</b> |
| <b>Anexo B. Licencias .....</b>                | <b>39</b> |
| B.1 Licencia del proyecto desarrollado .....   | 39        |
| B.2 Uso de proyectos de terceros .....         | 39        |
| B.3 Otras herramientas y dependencias .....    | 40        |

## Tabla de Ilustraciones

|                                                                                                                          |    |
|--------------------------------------------------------------------------------------------------------------------------|----|
| Ilustración 1: Visión general del sistema de automatización de despliegues CI/CD en entornos Cloud. ....                 | 3  |
| Ilustración 2: Motores y entornos de bases de datos.....                                                                 | 4  |
| Ilustración 3: Flujo simplificado de un pipeline CI/CD desde el desarrollo hasta el despliegue en AWS. ....              | 11 |
| Ilustración 4: Ciclo DevOps que integran las fases de desarrollo, integración, despliegue y operación del software. .... | 11 |
| Ilustración 5: Tablero Trello. ....                                                                                      | 14 |
| Ilustración 6: Una de las tarjetas del tablero Trello. ....                                                              | 14 |
| Ilustración 7: Etiquetas para las tarjetas Trello.....                                                                   | 15 |
| Ilustración 8: Casos de GitHub Actions. ....                                                                             | 16 |
| Ilustración 9: Versiones de la aplicación principal. ....                                                                | 17 |
| Ilustración 10: Contenedores Docker.....                                                                                 | 18 |
| Ilustración 11: Funciones que realiza SonarCloud. ....                                                                   | 20 |
| Ilustración 12: MVC de Spring Boot.....                                                                                  | 21 |
| Ilustración 13: Arquitectura hexagonal.....                                                                              | 22 |
| Ilustración 14: Diagrama de la arquitectura por capas.....                                                               | 23 |
| Ilustración 15: Muestra el flujo completo del asistente de despliegues y el control de validación por pasos. ....        | 24 |
| Ilustración 16: Integración de CI/CD.....                                                                                | 25 |
| Ilustración 17 Diagrama de las bases de datos. ....                                                                      | 27 |
| Ilustración 18: Manuales para el usuario. ....                                                                           | 30 |
| Ilustración 19: certificación AWS Certified Cloud Practitioner.....                                                      | 36 |
| Ilustración 20: certificación AWS Certified Solutions Architect- Associate.....                                          | 36 |
| Ilustración 21: Vista simplificada de la estructura del proyecto. ....                                                   | 38 |

## Contenido de tablas

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Tabla 1: objetivos técnicos .....                                              | 8  |
| Tabla 2: variables utilizadas para la integración con servicios externos ..... | 29 |

# 1. INTRODUCCIÓN

En este apartado daremos a conocer el contexto del proyecto, su motivación y aquellos aspectos que permiten comprender el funcionamiento de la solución propuesta.

## 1.1 Contexto del Proyecto

La evolución en el desarrollo de software ha experimentado grandes cambios en las infraestructuras Cloud, las cuales ayudan a permitir despliegues de aplicaciones de forma rápida, segura y escalable. Esto tiene como problema que los procesos tradicionales de despliegue manuales no sean del todo suficientes en muchos entornos profesionales.

Actualmente son más relevantes la cultura empresarial DevOps y los procesos de integración y despliegue continuos. DevOps se trata de una combinación de planteamientos, prácticas y herramientas que ayudan a la empresa a acelerar en gran medida el proceso de creación y aplicación de software y, a la vez, mejorar su calidad ([IONOS, s. f.](#)).

El uso de servicios Cloud ha facilitado el acceso a infraestructuras complejas sin necesitar una gestión directa del hardware, permitiendo a los desarrolladores centrarse en el desarrollo del software. Esta flexibilidad hace una mayor complejidad en la configuración y gestión de los despliegues, especialmente para perfiles con menos experiencia en entornos Cloud.

Como consecuencia, nace la necesidad de desarrollar soluciones que ayuden a simplificar y estructurar los procesos de despliegue, creando herramientas de ayuda que guíen al usuario a optimizar su tiempo y evitar posibles errores. Este TFG desarrolla la automatización y gestión de despliegues como un elemento clave dentro del ciclo de vida del software.

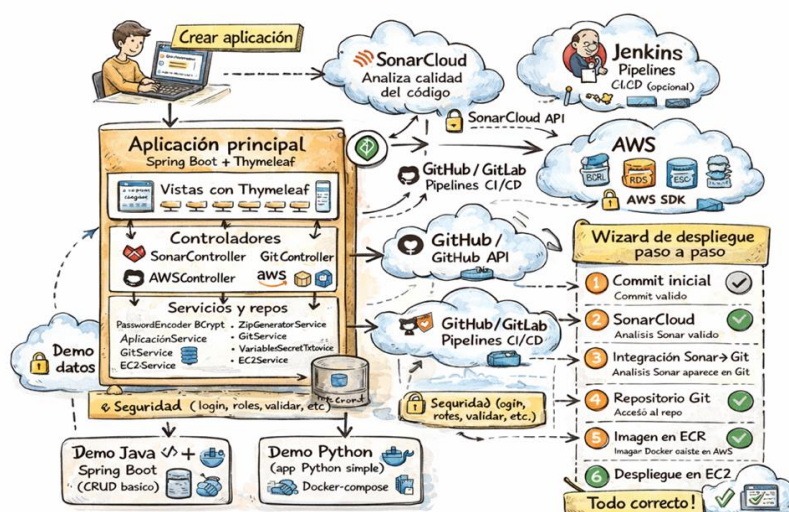


Ilustración 1: Visión general del sistema de automatización de despliegues CI/CD en entornos Cloud.

La Ilustración 1 muestra una visión global del sistema desarrollado en este TFG. En ella se representan los principales componentes de la aplicación web, desarrollada con Spring Boot y Thymeleaf, así como su integración con herramientas externas utilizadas en el proceso de automatización del despliegue.

El sistema interactúa con diferentes herramientas de CI/CD, como GitHub, GitLab o Jenkins, permitiendo automatizar procesos como la validación del código, la construcción de la aplicación y su posterior despliegue. Además, integra servicios de análisis de calidad del código mediante SonarCloud, así como servicios de Amazon Web Services (AWS) para realizar el despliegue final de las aplicaciones generadas mediante contenedores Docker.

El asistente también permite seleccionar distintos motores de bases de datos, tanto relacionales como MySQL o PostgreSQL, como no relacionales, como MongoDB, pudiendo configurarse en modo local o remoto según las necesidades del despliegue. Si se elige la opción de remoto, se pueden utilizar servicios como Atlas (MongoDB) o RDS que es un servicio de base de datos de AWS. Esas dos opciones son las que se han utilizado para realizar las pruebas oportunas.



*Ilustración 2: Motores y entornos de bases de datos.*

Finalmente, la Ilustración 2 muestra el funcionamiento general del asistente de despliegue guiado, que organiza el proceso en diferentes pasos secuenciales que el usuario debe completar para conseguir el despliegue automatizado de la aplicación.



## 1.2 Motivación

La motivación principal es ayudar a los usuarios con menos experiencia o en proceso de formación a realizar despliegues de una forma correcta, limpia y optimizada. La gente se pasa mucho tiempo buscando como implementar el despliegue de una forma automatizada. Por eso surge esta Aplicación Web, para guiarlos de una forma interactiva y conseguir su despliegue en poco tiempo.

Este proyecto muestra el interés por introducirnos en el uso de servicios Cloud y en la adopción del paradigma empresarial DevOps, especialmente en la parte de AWS. El TFG no solo es un objetivo académico, sino que además busca servir como inicio de aprendizaje práctico y como preparación para un entorno profesional.

## 1.3 Justificación del trabajo

La opción de elegir este TFG se justifica por la necesidad de demostrar de forma práctica los conocimientos y habilidades adquiridas en las diferentes áreas.

Desde el punto de vista académico, el proyecto permite evaluar competencias como el análisis de requisitos, el diseño de soluciones software, la toma de decisiones técnicas y la capacidad de insertar diferentes tecnologías en un mismo sistema.

## 1.4 Alcance del proyecto

El alcance del TFG se enfoca en el diseño y desarrollo de una aplicación web orientada a la gestión y automatización del proceso de despliegue de aplicaciones en entornos Cloud. Este proyecto guía al usuario para cubrir todas las fases necesarias para un despliegue, desde la configuración inicial de una aplicación hasta la generación de los archivos de configuraciones necesarios para conseguir su despliegue automatizado.

En base a lo expuesto anteriormente el proyecto es viable y concuerda con los objetivos académicos planteados.

## 1.5 Materiales y entregables

El conjunto de entregables se conforma por una serie de materiales:

### **Repositorio del proyecto**

Se proporciona el repositorio del proyecto alojado en GitHub, que contiene el código fuente de la aplicación, así como los recursos necesarios para su ejecución y despliegue.



El proyecto se estructura como un repositorio que incluye la aplicación principal desarrollada en Spring Boot, junto con proyectos de demostración y los ficheros necesarios para la automatización del despliegue.

### **1) Manual de uso y documentación para el usuario**

Dentro del repositorio se incluyen los manuales de cada paso del wizard en formato PDF, que explican el funcionamiento del sistema y guían al usuario. Estos manuales se encuentran disponibles en el directorio:

tfg-cicd-aws-2526/app/src/main/resources/static/manuales

### **2) Documentación técnica del código (JavaDoc)**

El proyecto incluye documentación técnica generada mediante JavaDoc, que describen las clases, métodos y componentes del sistema. Con esta documentación facilitamos el análisis del código.

### **3) Documentación del repositorio**

El repositorio incluye un archivo README.md que describe el propósito del proyecto, las tecnologías empleadas, su arquitectura y los pasos necesarios para ejecutar o desplegar una aplicación con la configuración generada.

### **4) Memoria del TFG**

Se entrega la memoria del proyecto, que describe el contexto, los objetivos, las decisiones de diseño, la arquitectura del sistema, las tecnologías utilizadas y la validación del sistema desarrollado.

### **5) Documentación técnica complementaria (anexo)**

Esta documentación técnica amplía la información de la memoria, incluyendo:

- Plan de proyecto software.
- Especificaciones de requisitos.
- Diseño del sistema.
- Documentación técnica de programación.
- Manual de usuario
- Anexo de sostenibilidad curricular.

Este documento sirve de complemento a la memoria principal, proporcionando una visión más detallada del desarrollo del sistema.

## 2. OBJETIVOS DEL PROYECTO

El objetivo general es diseñar y desarrollar una aplicación web que haga de asistente de despliegues, ayudando a la creación, configuración y automatización del despliegue de aplicaciones en entornos Cloud, teniendo en cuenta la metodología empresarial DevOps.

### 2.1 Objetivos funcionales

Los objetivos funcionales de este TFG son los siguientes:

- Permitir gestionar y realizar despliegues de aplicaciones utilizando la metodología CI/CD a través de un interfaz web.
- Guiar al usuario mediante un asistente de despliegues estructurado por pasos secuenciales.
- Permitir la selección de la herramienta de integración.
- Facilitar la elección entre los diferentes motores de bases de datos (relacional o no relacional) y su modo (local o remoto)
- Gestionar la configuración de las aplicaciones en función del entorno de despliegue.
- Generar los archivos y recursos para facilitar el despliegue automatizado.
- Facilitar manuales y guías de apoyo.
- Dar la posibilidad al usuario de descargar un proyecto ejemplo (demo) o solo los archivos de configuración para los despliegues.

### 2.2 Objetivos técnicos

Conforma a la Ilustración 1, el sistema desarrollado integra diferentes tecnologías y herramientas necesarias para la automatización de despliegues en entornos Cloud.

La Tabla 1 resume los principales componentes técnicos utilizados en el desarrollo de la aplicación o en las soluciones que permite desplegar, así como su función dentro del sistema.

| Área del sistema                      | Tecnologías Utilizadas                                | Función principal                                              |
|---------------------------------------|-------------------------------------------------------|----------------------------------------------------------------|
| <b>Backend y arquitectura</b>         | Spring Boot 3.4.5<br>Spring MVC<br>Thymeleaf 3.1.3    | Desarrollo de la aplicación web y del asistente de despliegues |
| <b>Acceso a datos</b>                 | Spring Data JPA<br>Hibernate 6.6.13<br>Flyway 10.20.1 | Gestión de entidades, consultas y migraciones de base de datos |
| <b>Seguridad</b>                      | Spring Security 6.4.5                                 | Autenticación y protección de recursos                         |
| <b>Bases de datos relacionales</b>    | PostgreSQL<br>Mysql                                   | Configuración de entornos SQL                                  |
| <b>Bases de datos no relacionales</b> | MongoDB                                               | Configuración de entornos no SQL                               |
| <b>CI/CD</b>                          | Github Actions<br>GitLab CI/CD<br>Jenkins             | Automatización de pipelines y despliegues                      |
| <b>Contenedores</b>                   | Docker<br>Docker Compose                              | Empaquetado y orquestación de servicios                        |
| <b>Infraestructura cloud</b>          | AWS EC2<br>AWS ECR                                    | Despliegue y ejecución de aplicaciones                         |
| <b>Calidad del código</b>             | SonarCloud                                            | Análisis estático y validación de calidad                      |
| <b>Control de versiones</b>           | GitHub                                                | Gestión y versionado del código fuente                         |

Tabla 1: objetivos técnicos

## 2.3 Objetivos académicos

Los objetivos académicos de este TFG son los siguientes:

- Aplicar de forma práctica los conocimientos adquiridos a lo largo del grado en un proyecto real.
- Consolidar conceptos relacionados con el desarrollo de Cloud Computing, software y DevOps.
- Entender el ciclo completo de vida de una aplicación, incluyendo la fase de desarrollo y despliegue.
- Implementar distintas tecnologías y herramientas dentro de una solución software coherente.
- Obtener experiencia en la planificación y gestión de un proyecto software.
- Mejorar la capacidad de documentación y comunicación técnica.

## 3. CONCEPTOS TEÓRICOS

En este apartado se presentan los principales conceptos teóricos relacionados con el desarrollo del proyecto. Se introducen ideas fundamentales como el enfoque DevOps, la integración y despliegue continuo, la gestión de configuraciones o el uso de entornos Cloud. Estos conceptos sirven como base para comprender las decisiones técnicas y el funcionamiento del sistema desarrollado.

### 3.1 DevOps como enfoque de cultura empresarial

El enfoque DevOps nace para promover la colaboración entre los equipos de desarrollo y las operaciones a lo largo del ciclo de vida del software, favoreciendo la automatización de procesos y la mejora continua en el desarrollo y despliegue de aplicaciones ([Kim et al., 2016](#)). DevOps se utiliza como una forma de trabajo orientada a mejorar la comunicación, automatización y la eficiencia en los procesos de desarrollo y despliegue.

Algunos de los principales fundamentos de la metodología DevOps se encuentran en la automatización de tareas repetitivas, integración continua, entrega frecuente de software y la monitorización constante de los sistemas. Estos principios nos ayudan a reducir errores, hacer entregas a tiempo y mejorar la calidad del software.

## 3.2 Separación de entornos y gestión de configuración

Es frecuente trabajar con distintos entornos, como desarrollo, integración o producción, cada uno con sus funciones propios. Tener una separación correcta de estos entornos hace más fácil evitar errores en el uso de configuraciones inadecuadas y para garantizar un comportamiento adecuado en cada fase de la aplicación.

La gestión de la configuración permite definir y controlar los recursos necesarios para la ejecución de una aplicación en cada entorno, como variables de entorno, credenciales o ajustes específicos.

## 3.3 Integración y despliegue continuo (CI/CD)

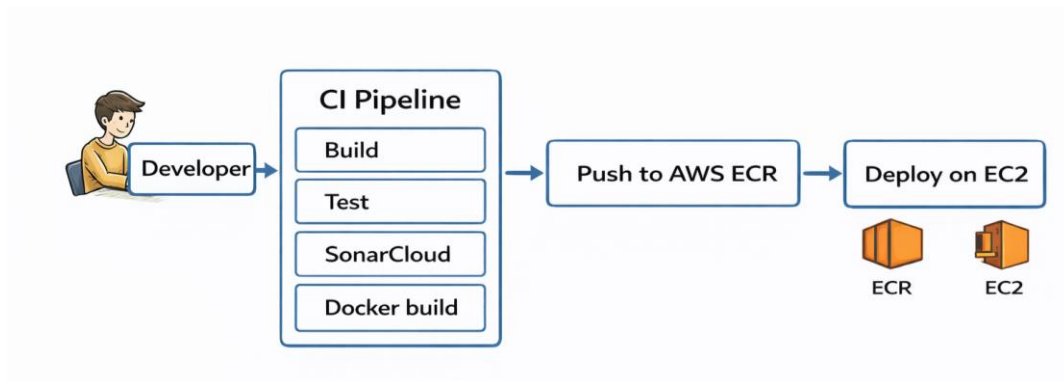
La integración continua propone integrar cambios de código de forma frecuente en un repositorio común, permitiendo su validación automática mediante pruebas y comprobaciones básicas ([Fowler, 2006](#)). Los errores se pueden identificar en fases tempranas y así poder disminuirlos en fases posteriores.

El despliegue continuo nos sirve para automatizar el proceso de entrega del software hasta los entornos de ejecución, una vez que se superan las fases de validación. Esto permite una entrega rápida y repetible de nuevas versiones, minimizando las intervenciones manuales y favorecer la fiabilidad. La combinación de los procesos anteriores constituye uno de los pilares de la cultura de empresa DevOps, ya que se consiguen flujos de trabajo más eficientes, predecibles y controlados.

Estos procesos suelen configurarse mediante pipelines automatizados, donde cada cambio en el repositorio desencadena una serie de etapas que validan el código, generan los artefactos necesarios y realizan el despliegue en la infraestructura de destino.

Desde el punto de vista del despliegue, la aplicación se empaqueta como una imagen Docker, lo que permite garantizar la portabilidad y consistencia del entorno de ejecución. Esta imagen se almacena en Amazon Elastic Container Registry (ECR), desde donde posteriormente es descargada y desplegada en una instancia Amazon EC2 dentro de la infraestructura de Amazon Web Services (AWS), que proporciona servicios Cloud que permiten desplegar aplicaciones de forma flexible y escalable ([Amazon Web Services, 2023](#)).

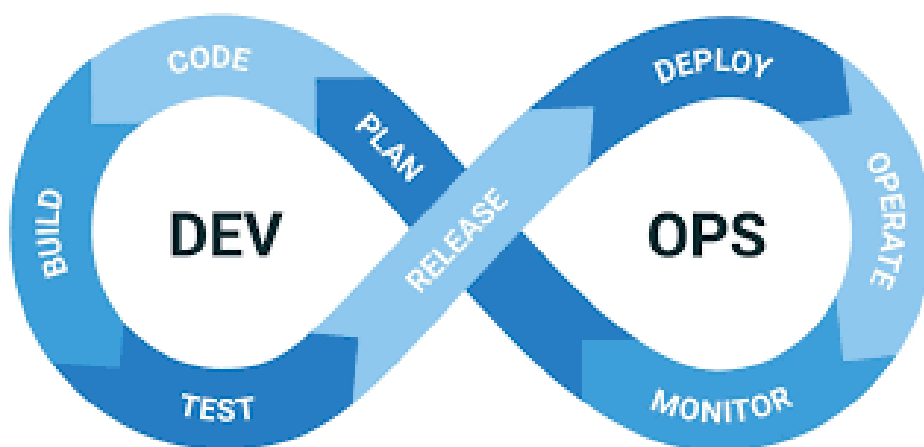
La Ilustración 3 muestra un ejemplo simplificado del flujo de un pipeline CI/CD utilizado en el proyecto.



*Ilustración 3: Flujo simplificado de un pipeline CI/CD desde el desarrollo hasta el despliegue en AWS.*

En este flujo, cuando un desarrollador realiza un commit del main o un tag ([ver detalles en el apartado 3.2.3](#)), se activa automáticamente el pipeline de integración continua. Durante este flujo se ejecutan distintas etapas, como la compilación del proyecto (Build), la ejecución de pruebas (test), el análisis de calidad mediante SonarCloud y la construcción de la imagen Docker necesaria para el despliegue automatizado de la aplicación.

La separación de entornos forma parte de una práctica habitual de los procesos DevOps, donde se distinguen claramente las fases de desarrollo y operación del software.



*Ilustración 4: Ciclo DevOps que integran las fases de desarrollo, integración, despliegue y operación del software.*

Tal y como se muestra en la figura, el ciclo DevOps divide el proceso en dos grandes bloques:

- Fase de desarrollo (DEV)→ incluye las etapas de planificación, desarrollo, construcción y pruebas. Durante esta fase los desarrolladores implementan nuevas funcionalidades, realizan pruebas y validan el correcto funcionamiento de la aplicación en un entorno controlado.

- Fase de producción (OPS)→ comprende las etapas de despliegue, monitorización y operación del sistema. En esta fase la aplicación se ejecuta en un entorno de producción, donde se gestionan los recursos necesarios para su funcionamiento y se supervisa su comportamiento.

### 3.4 Automatización de despliegues

El despliegue de aplicaciones es una fase crítica dentro del ciclo de vida del software, cuando se hace manualmente da lugar a errores y problemas de consistencia. La necesidad de repetir tareas y gestionar configuraciones específicas aumenta su complejidad y dificulta la ejecución fiable del despliegue.

Ejecutarlo automáticamente permite abordar esta situación estableciendo flujos de despliegues definidos y repetibles, optimizando el tiempo dedicado a tareas operativas. Esto resulta esencial para lograr entregas de software controladas, estables y rápidas.

### 3.5 Asistente de despliegue de entornos Cloud

El Cloud Computing permite ofrecer recursos informáticos bajo demanda a través de internet, facilitando el acceso a infraestructuras escalables sin necesidad de gestionar directamente el hardware ([Mell & Grance, 2011](#)).

El despliegue en entornos Cloud suele contener configuraciones de múltiples elementos, como servicios, variables de entorno, credenciales y recursos asociados.

El asistente de despliegue surge para guiar a usuarios a través de un proceso, estructurando las diferentes fases y disminuyendo la necesidad de tener experiencia sobre despliegues. Este asistente permite crear configuraciones de forma progresiva, comprendiendo cada paso y minimizando errores.

Los asistentes de despliegues se utilizan para estandarizar procesos y mejorar la accesibilidad a este tipo de entornos. Esto hace que sea eficaz para combinar automatización, aprendizaje y buenas prácticas, haciendo posible a los usuarios diseñar el despliegue de aplicaciones en Cloud de forma más sencilla y controlada.

## 4. METODOLOGÍAS, HERRAMIENTAS Y TÉCNICAS.

En este apartado se describen las metodologías de trabajo, herramientas y tecnologías utilizadas durante el desarrollo del proyecto. Se explican los métodos de planificación y organización del trabajo, así como las herramientas empleadas para el desarrollo, control de versiones, despliegue y análisis de calidad del código.



## 4.1 Metodología

El desarrollo de la WebApp se ha llevado a cabo siguiendo una metodología de trabajo iterativa e incremental, que ha permitido realizar el proyecto de una forma progresiva y flexible. Esto ha facilitado la adaptación a los cambios que han surgido en el proceso de desarrollo y ha permitido comprobar de manera continua sus funcionalidades.

### 4.1.1 Planificación del proyecto

En función de las necesidades de cada etapa de desarrollo se han ido definiendo los diferentes objetivos a alcanzar. Esto ha permitido organizar el trabajo de manera estructurada y manteniendo una visión global del estado del proyecto en cada fase.

El final de cada fase se ha orientado a la consecución del Producto Mínimo Viable (MVP) para tener permanentemente una aplicación que cada vez incorporaba más funcionalidad pero que siempre estaba operativa.

Para la gestión y seguimiento de las tareas se ha utilizado Trello una herramienta de planificación visual. La herramienta ha permitido mejorar la gestión del tiempo y controlar el progreso del proyecto.

### 4.1.2 Uso de Trello para la gestión de tareas

Se ha utilizado la herramienta Trello, que ha servido de apoyo para la planificación de todo el proyecto de una forma visual. Ha facilitado definir las diferentes tareas y hacer el seguimiento del estado.

En la Ilustración 5 se ve como el proyecto se ha estructurado mediante un tablero con diferentes bloques marcados por estados como: más tarde/problemas, por hacer, en proceso, revisión o hecho. Esta organización ha permitido visualizar fácilmente el progreso de cada tarea a lo largo del desarrollo del proyecto.

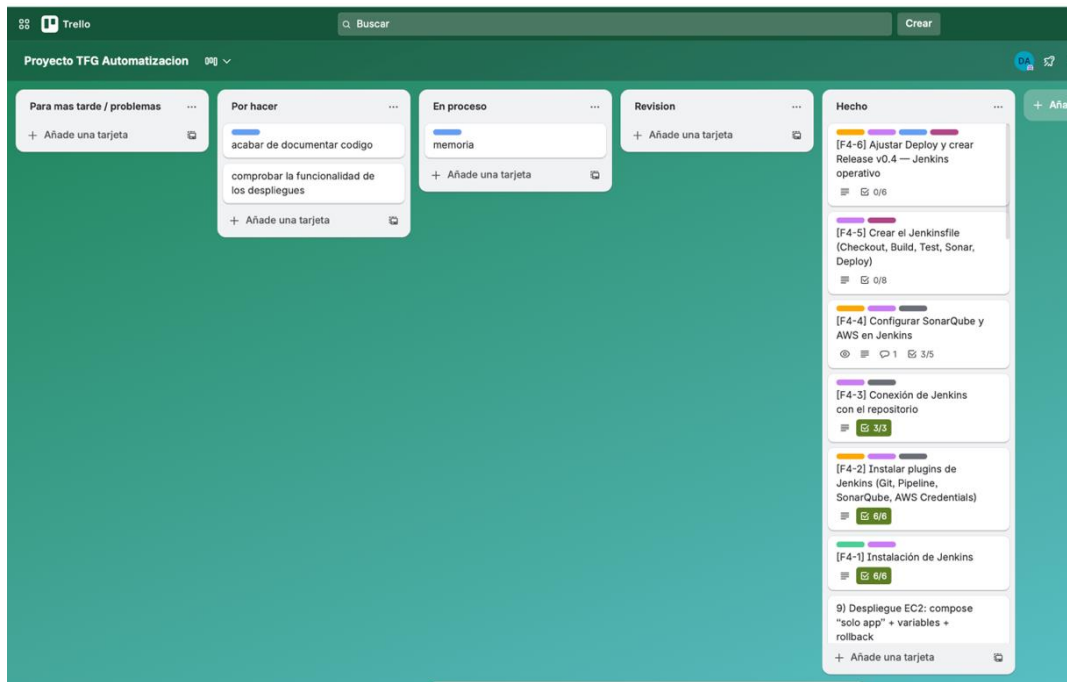


Ilustración 5: Tablero Trello.

En la Ilustración 6 se aprecia una tarea del proyecto, la cual se representa mediante una tarjeta, donde se detalla la funcionalidad o actividad a desarrollar. Dentro de cada tarjeta se han definido listas de verificación (checklists) que permiten dividir la tarea en subtarefas más pequeñas y comprobar su progreso de forma estructurada.

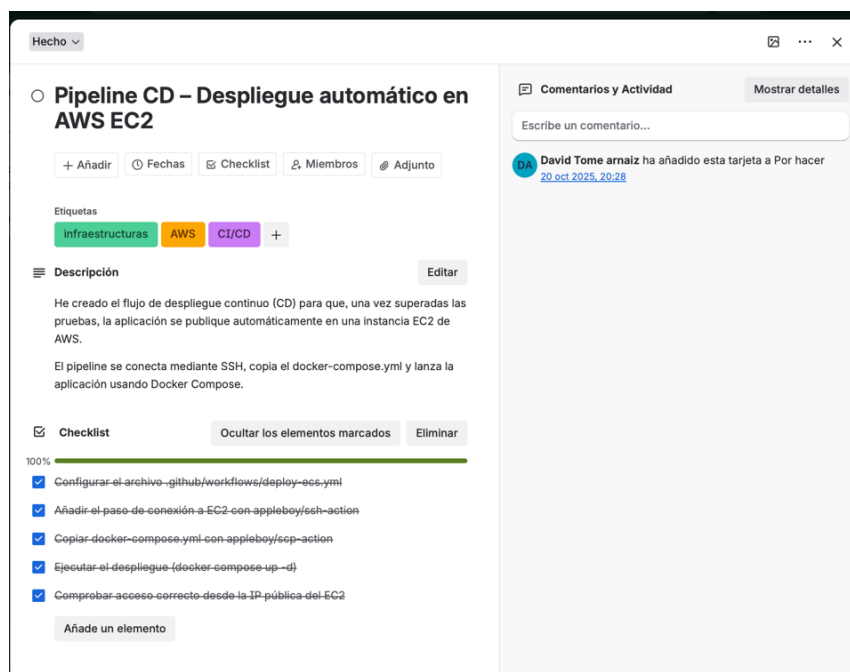


Ilustración 6: Una de las tarjetas del tablero Trello.

Además, en la Ilustración 7 se muestra cómo se han utilizado etiquetas de colores para clasificar las tareas según su temática o área del trabajo, como por ejemplo infraestructura, planificación, AWS, CI/CD o documentación. Este sistema de etiquetado permite identificar rápidamente el tipo de trabajo asociado a cada tarea y facilita la organización general del proyecto.

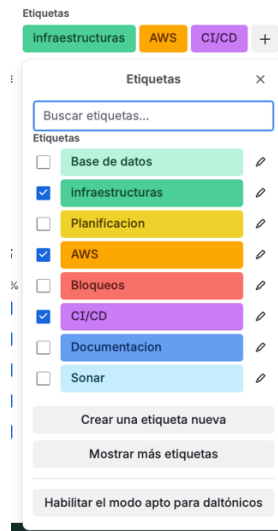


Ilustración 7: Etiquetas para las tarjetas Trello.

Gracias a esta estructura, Trello ha permitido fragmentar las tareas en objetivos más pequeños y controlables, facilitando el seguimiento del desarrollo y mejorando la organización del trabajo durante todo el proyecto. De esta manera, se ha conseguido mantener una planificación clara y un control continuo del avance del TFG.

## 4.2 Control de versiones

El control de versiones constituye un elemento fundamental en el desarrollo del proyecto, ya que permite gestionar de forma estructurada la evolución del código fuente a lo largo del tiempo. Para ello se ha utilizado el sistema de control de versiones Git, gestionado mediante la plataforma GitHub.

El uso de GitHub ha permitido registrar de manera detallada todas las modificaciones realizadas durante el desarrollo, facilitando el seguimiento de cambios, la recuperación de versiones anteriores y la organización incremental del proyecto. Esta metodología ha favorecido una implementación progresiva de las funcionalidades, permitiendo detectar errores y validar mejoras de forma controlada.

Además, la utilización de GitHub ha proporcionado un entorno centralizado del repositorio, manteniendo organizados tanto el código fuente como la documentación asociada al proyecto. De este modo, se garantiza la trazabilidad, estabilidad y mantenibilidad del sistema a lo largo del ciclo de

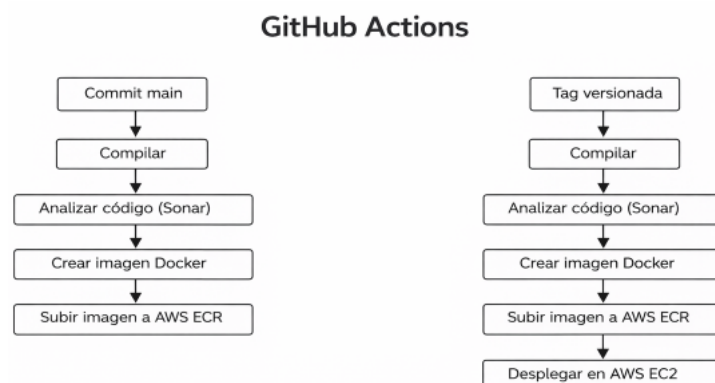
desarrollo, facilitando la gestión del código fuente y la colaboración en el desarrollo del proyecto mediante repositorios remotos ([GitHub, s. f.](#)).

#### 4.2.1 Uso de GitHub Actions

En la elaboración del proyecto se ha implementado GitHub Actions como herramienta de CI/CD. Github Actions permite definir flujos de integración y despliegue continuo mediante archivos de configuración que automatizan tareas de construcción, validación y despliegue del software ([GitHub, s. f.](#)).

Mediante la definición de workflows configurados en archivos YAML, se han establecidos flujos de trabajo automatizados que se ejecutan en dos casos (Ilustración 8):

1. **Cuando se hace un Commit en el main:** se compila, se hace el análisis del código con la herramienta de Sonar, se crea la imagen Docker y se almacena en el repositorio ECR dentro de AWS.
2. **Cuando se crea una etiqueta versionada:** se hace lo mismo que en el Commit del main, pero esta vez se despliega la nueva imagen Docker en la instancia EC2 de AWS.



*Ilustración 8: Casos de GitHub Actions.*

Esto ha permitido integrar de forma directa el control de versiones con el proceso de despliegue, mejorando la eficiencia y reduciendo posibles errores manuales.

#### 4.2.2 Organización del repositorio

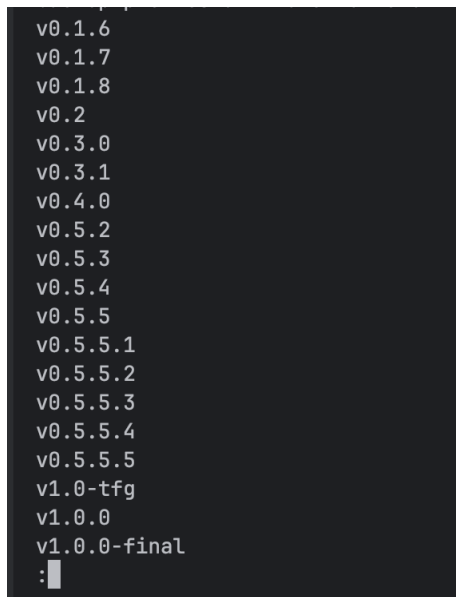
La organización del repositorio constituye un aspecto relevante durante el desarrollo del proyecto, debido a que una estructura clara y organizada facilita el mantenimiento del código y la comprensión del sistema. Para ello el repositorio se ha organizado separando los distintos elementos que conforman el proyecto.

### 4.2.3 Control de versiones en el proyecto

El control de versiones en el proyecto (tag) se ha utilizado como mecanismo para establecer las entregas del MVP durante las diferentes etapas de desarrollo. Esto ha ayudado tener un control más claro sobre la evolución del proyecto.

Para ello, se han generado varias versiones del proyecto, permitiendo reflejar la evolución progresiva de las funcionalidades.

La Ilustración 9 muestra las versiones etiquetadas de proyecto almacenadas en el repositorio GitHub, evidenciando la evolución desde las primeras iteraciones hasta la versión más estable.



```
v0.1.6  
v0.1.7  
v0.1.8  
v0.2  
v0.3.0  
v0.3.1  
v0.4.0  
v0.5.2  
v0.5.3  
v0.5.4  
v0.5.5  
v0.5.5.1  
v0.5.5.2  
v0.5.5.3  
v0.5.5.4  
v0.5.5.5  
v1.0-tfg  
v1.0.0  
v1.0.0-final  
:
```

*Ilustración 9: Versiones de la aplicación principal.*

## 4.3 Herramientas utilizadas

### 4.3.1 Framework de desarrollo: IntelliJ IDEA

El desarrollo del proyecto se ha llevado a cabo utilizando el Framework de desarrollo integrado IntelliJ IDEA, una herramienta ampliamente utilizada en el ámbito profesional para la creación de aplicaciones basadas en Java y frameworks como Spring Boot.

IntelliJ IDEA ha permitido gestionar el proyecto de manera estructurada, facilitando la organización de paquetes, clases y dependencias mediante la integración con Maven. Entre sus principales ventajas destaca el autocompletado inteligente de código, la detección de errores en tiempo real, la refactorización automática y las herramientas de depuración, que han contribuido a mejorar la calidad del desarrollo.

Además, el entorno proporciona una integración directa con servidores embebidos como Tomcat, lo que ha permitido ejecutar y probar la aplicación de manera local durante las distintas fases del desarrollo. También ha facilitado la gestión de configuraciones del proyecto y la conexión con repositorios de control de versiones.

El uso de IntelliJ IDEA ha contribuido a mantener una estructura organizada del proyecto, optimizar el tiempo de desarrollo y aplicar buenas prácticas en la implementación del backend basado en Spring Boot.

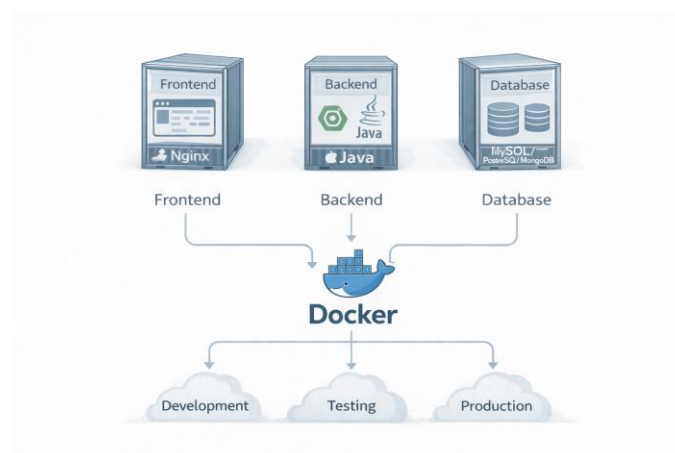
### 4.3.2 Contenedores

Docker permite empaquetar aplicaciones junto con todas sus dependencias dentro de contenedores aislados, garantizando consistencia entre los diferentes entornos de ejecución ([Docker, s. f.](#)). Esta característica es importante en proyectos como el nuestro que integran múltiples componentes donde se combinan un backend desarrollado con Spring Boot, bases de datos configurables y procesos de automatización mediante herramientas CI/CD.

En el contexto de aplicaciones con backend y frontend, Docker facilita la separación de responsabilidades mediante la creación de contenedores independientes para cada servicio, tal y como se muestra en la Ilustración 10.

Asimismo, Docker se integra de forma natural con los procesos de CI/CD, permitiendo automatizar la construcción de imágenes, su almacenamiento en repositorios y su posterior despliegue en entornos Cloud como en nuestro caso AWS. De este modo, el uso de contenedores refuerza el objetivo del proyecto: simplificar y estructurar el proceso de despliegue de aplicaciones.

La adopción de Docker en el proyecto ayuda a reducir problemas derivados de incompatibilidades de entorno, facilita la replicación del sistema en distintos escenarios y mejora la eficiencia del proceso de despliegue automatizado.



*Ilustración 10: Contenedores Docker.*

### 4.3.3 Bases de datos

PostgreSQL es un sistema de gestión de bases de datos relacional ampliamente utilizado en aplicaciones modernas debido a su estabilidad, rendimiento y compatibilidad con diferentes entornos de desarrollo ([PostgreSQL Global Development Group, s. f.](#)).

En este proyecto, PostgreSQL se utiliza como motor de base de datos principal para gestionar la información necesaria para el funcionamiento de la aplicación. Su integración con Spring Boot y Spring Data JPA permite gestionar de forma eficiente el acceso y persistencia de los datos.

Además, durante el desarrollo local del proyecto, la base de datos PostgreSQL se ejecuta mediante contenedores Docker, facilitando la replicación del entorno y la configuración del sistema.

#### 4.3.3.1 Uso de Flyway

En el proyecto se utiliza Flyway como herramienta de gestión de migraciones para la base de datos de la aplicación principal, que se encarga de almacenar los datos necesarios para el funcionamiento del asistente de despliegues.

Flyway ha permitido versionar y aplicar de forma controlada cambios en la base de datos del sistema, ayudando a su evolución a lo largo del desarrollo y garantizando la coherencia del modelo de datos entre distintos entornos de ejecución.

El uso de migraciones automatizadas durante el desarrollo es muy útil, ya que permite inicializar y actualizar la base de datos sin necesidad de intervenir manualmente.

Flyway no se aplica en el proceso de despliegue, ya que el objetivo del sistema es generar la configuración necesaria para su ejecución.

### 4.3.4 Calidad del código

La calidad del código es un aspecto importante a la hora de desarrollar aplicaciones software, sobre todo en proyectos que desean aplicar buenas prácticas. Tener un código legible, estructurado y mantenible hace que se disminuyan los errores y facilita la evolución de la aplicación a lo largo del tiempo.

En este proyecto se ha utilizado la herramienta SonarCloud con el objetivo de evaluar su calidad, para detectar posibles errores, malas prácticas o problemas de mantenibilidad como se muestra en la Ilustración 11. Estos análisis hacen que podamos corregir errores en etapas tempranas y ayudarnos a un desarrollo más controlado.

El análisis de calidad refuerza la parte de integración continua (CI) del proyecto, ya que permite automatizar procesos de validación y mejora del código durante el desarrollo. De este modo, la calidad se integra dentro del ciclo de vida de la aplicación, contribuyendo una solución más robusta y mantenible.

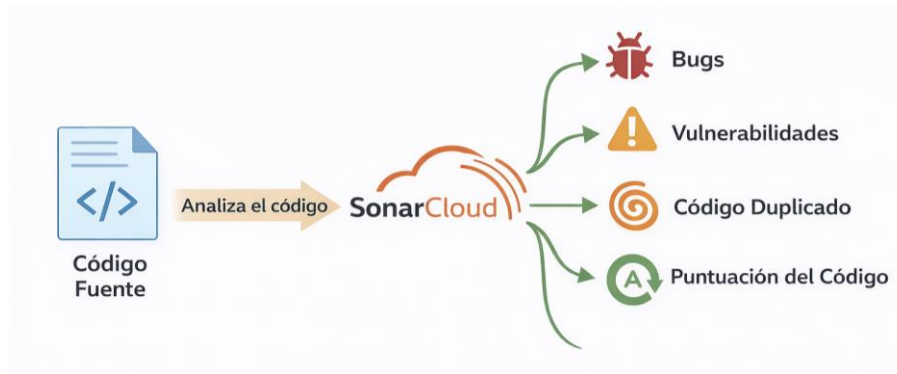


Ilustración 11: Funciones que realiza SonarCloud.

## 5. ASPECTOS RELEVANTES DEL DESARROLLO DEL PROYECTO

En este apartado se describen los aspectos más relevantes relacionados con el desarrollo del sistema. Se analizan elementos como la arquitectura de la aplicación, el diseño del asistente de despliegues, la integración con herramientas de CI/CD, la gestión de bases de datos y los mecanismos de seguridad implementados.

### 5.1 Decisiones técnicas

A la hora de seleccionar las tecnologías a utilizar se han tenido en cuenta criterios como la adecuación de los objetivos planteados, la facilitación de integración entre herramientas y el uso de tecnologías en entornos profesionales. Esto ha ayudado a construir una solución flexible, coherente.

Para el desarrollo del backend se ha optado por el framework Spring Boot, basado en el ecosistema Spring, que facilita el desarrollo de aplicaciones Java, especialmente aplicaciones web y servicios backend. Este framework simplifica la configuración y puesta en marcha de los proyectos mediante mecanismos de configuración automática y una arquitectura estructurada ([Spring, s. f.](#)).

La aplicación se ha desarrollado siguiendo el patrón MVC, utilizando Spring MVC y Thymeleaf para separar la lógica de negocio, la gestión de peticiones y la representación de la interfaz de usuario.

Los componentes principales de esta arquitectura se corresponden con los mostrados en la Ilustración 12: MVC de Spring Boot.



## Modelo MVC – Asistente de Despliegues CI/CD en AWS

Aplicación Web Spring Boot + Thymeleaf, JPA/Hibernate

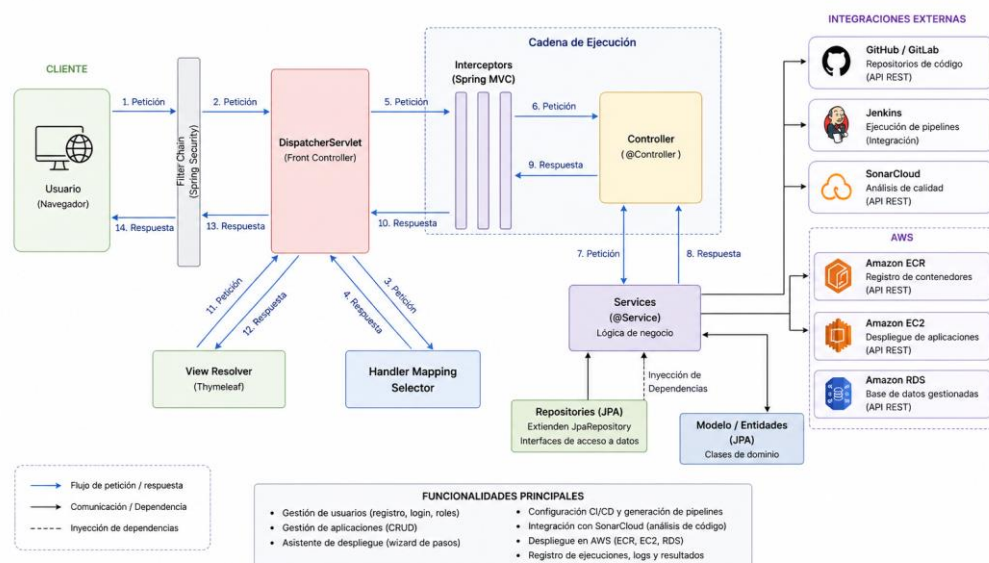


Ilustración 12: MVC de Spring Boot.

La arquitectura se complementa con diferentes integraciones externas y servicios Cloud que permiten ampliar las capacidades del sistema y automatizar distintos procesos relacionados con el despliegue y gestión de aplicaciones.

La utilización de esta arquitectura permite desarrollar una aplicación estructurada, mantenible y escalable, siguiendo buenas prácticas de separación de responsabilidades y organización por capas. Para el despliegue del sistema se emplea Docker Compose.

## 5.2 Arquitectura del sistema

En esta sección se describe la arquitectura general del sistema desarrollado en el proyecto. Se presenta una visión global de los componentes que forman parte de la aplicación, así como la forma en la que estos interactúan entre sí. Asimismo, se explica el entorno de despliegue utilizado y las tecnologías empleadas para ejecutar la aplicación dentro de la infraestructura Cloud.

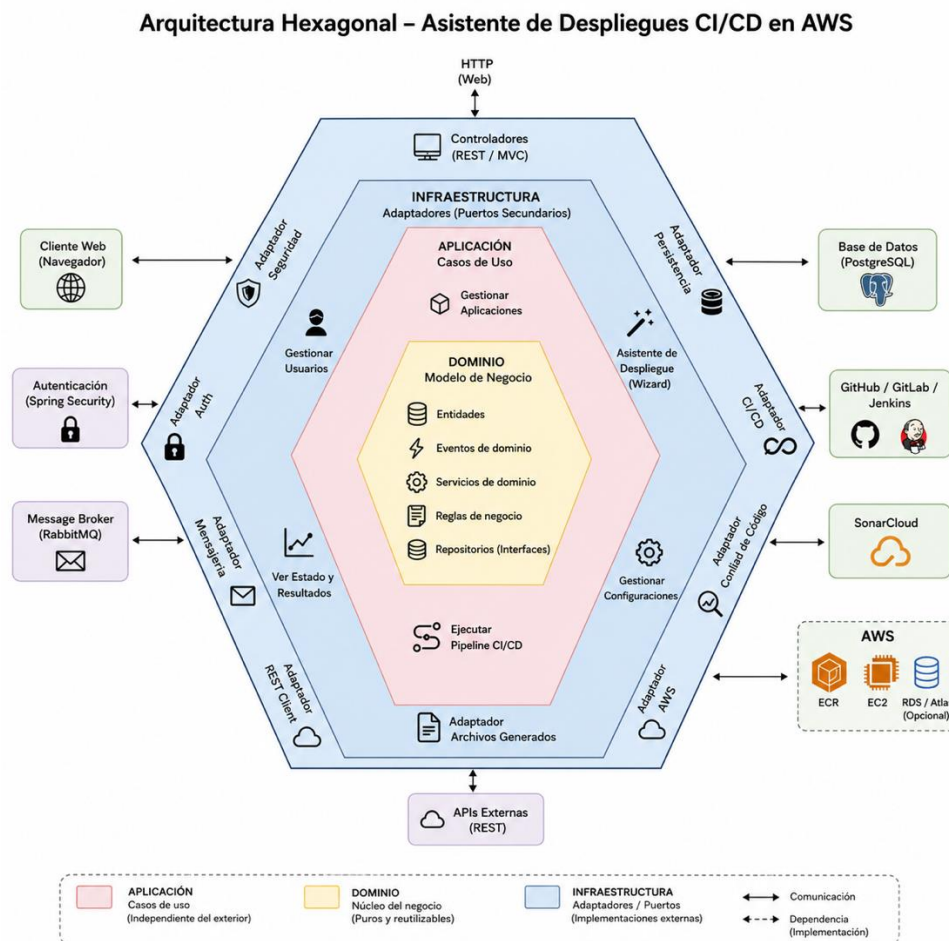
### 5.2.1 Arquitectura hexagonal

El sistema se ha diseñado siguiendo los principios de la arquitectura hexagonal (también conocida como arquitectos de puertos y adaptadores), cuyo objetivo es desacoplar la lógica de negocio de las dependencias externas.

Como podemos apreciar en la Ilustración 13, el núcleo del sistema (dominio y casos de uso) se mantiene independiente de elementos externos como bases de datos, servicios cloud o herramientas de integración continua.

Las interacciones con estos sistemas se realizan a través de adaptadores, que permiten conectar el sistema con tecnologías externas como AWS (EC2, ECR, RDS), herramientas de CI/CD (Github, Gitlab, Jenkins) y servicios de análisis de código como SonarCloud.

Este diseño mejora la mantenibilidad, facilita la evolución del sistema y permite sustituir fácilmente las tecnologías externas sin afectar al núcleo de la aplicación.



*Ilustración 13: Arquitectura hexagonal*

## 5.2.2 Diagrama funcional de arquitectura

Con el objetivo de mostrar en detalle la parte correspondiente al dominio, se ha decidido realizar el diagrama de arquitectura de la Ilustración 14.

Este diagrama representa los principales elementos que intervienen en el funcionamiento del sistema y como se organizan en diferentes capas. Se muestran la capa de presentación, la capa de negocio y la capa de datos, así como su interacción con servicios externos.

En la capa de presentación se encuentran los controladores y las vistas, que gestionan la interacción con el usuario a través del navegador web. La capa de negocio contiene la lógica principal de la aplicación, implementada mediante distintos servicios encargados de la gestión de usuarios, aplicaciones, configuraciones, integración con CI/CD y despliegue en AWS.

La capa de datos se encarga de la persistencia de la información, utilizando repositorios y una base de datos PostgreSQL, donde se almacenan entidades como usuarios, aplicaciones, configuraciones y registros de ejecución.

Finalmente, el sistema se integra con distintos servicios externos como Github/Gitlab, Jenkins, SonarCloud y AWS, que permiten automatizar el proceso de desarrollo, análisis de calidad y despliegue de aplicaciones.

Este enfoque separa las responsabilidades entre las distintas capas del sistema, facilitando su mantenimiento, escalabilidad y evolución.

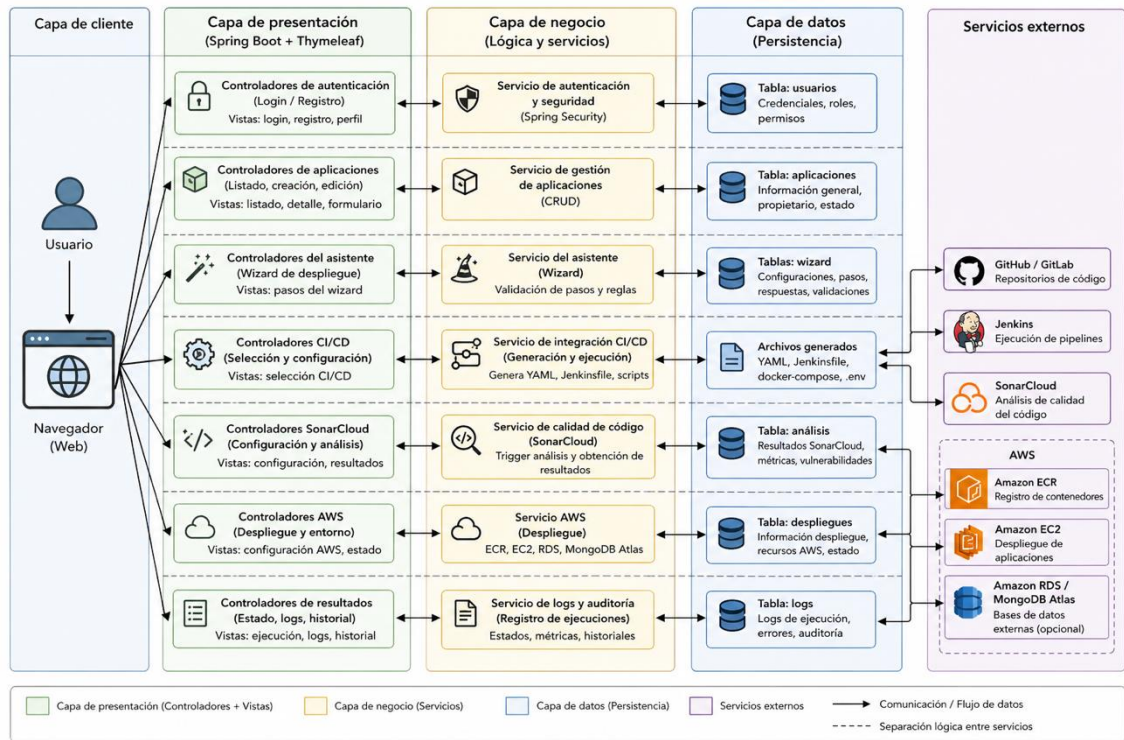


Ilustración 14: Diagrama de la arquitectura por capas.

### 5.3 Diseño del asistente de despliegues

Es la principal funcionalidad de la aplicación y tiene como objetivo guiar al usuario de una forma progresiva a través de las diferentes fases necesarias para automatizar el despliegue de cualquier aplicación en entornos Cloud, disminuyendo la complejidad de los procesos y minimizando errores. Para ello, el asistente integra las tecnologías y herramientas descritas en la Tabla 1: objetivos técnicos

El diagrama de flujo del asistente se muestra en la Ilustración 15:

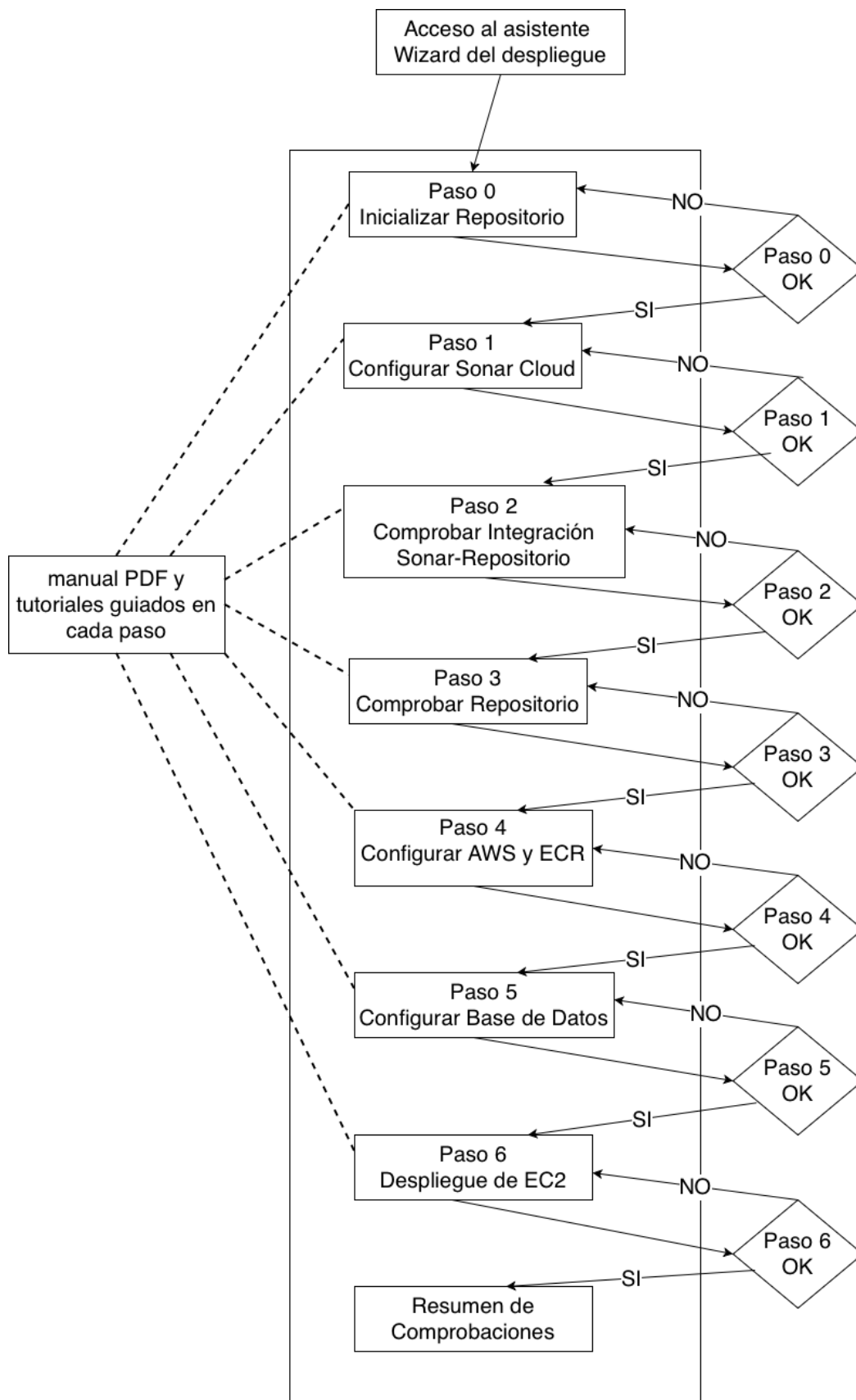


Ilustración 15: Muestra el flujo completo del asistente de despliegues y el control de validación por pasos.

### 5.3.1 Decisiones de diseño

Una de las principales decisiones de diseño ha sido insertar un mecanismo de validación por pasos. En cada fase del asistente, el usuario debe rellenar los datos necesarios. Este enfoque hace que disminuyan los errores producidos por una configuración incompleta o incorrecta.

Además, se ha decidido implementar soporte continuo para el usuario durante todo el proceso mediante manuales en formato PDF y tutoriales guiados añadidos en cada paso.

El asistente se ha diseñado para generar diferentes aplicaciones desplegadas en Amazon, permitiendo obtener proyectos de demostración completos o únicamente los archivos de configuración necesarios.

El proyecto genera configuraciones separadas en entorno mediante los ficheros `application-*.properties`, esto nos ayuda a ajustar los parámetros como la conexión a la base de datos, variables sensibles y puertos sin modificar código.

## 5.4 Integración CI/CD

La integración de herramientas de CI/CD (Jenkins, Github o Gitlab) constituye uno de los elementos clave del sistema desarrollado. El sistema permite incorporar el uso de diferentes herramientas CI/CD, de modo que el usuario puede elegir la opción que mejor se desempeñe a su proyecto.

Durante la generación del repositorio y el despliegue de la aplicación de demo, el usuario debe seleccionar la herramienta CI/CD que utilizará para el despliegue, quedando asociada a la aplicación y utilizada en el asistente. En función de la herramienta elegida, el sistema generará los archivos de configuración necesarios, garantizando que el proceso sea coherente con la herramienta seleccionada.

La integración de CI/CD en el asistente ayuda a automatizar tareas como la ejecución de análisis de calidad, la construcción de imágenes y el despliegue en la infraestructura Cloud.



Ilustración 16: Integración de CI/CD.



### 5.4.1 Generación de configuración de despliegue

La generación de la configuración del despliegue se hace de forma automática a partir de la información introducida en el formulario de la configuración y despliegue de la aplicación demo y la ejecución del asistente. El sistema utiliza estos datos para construir los recursos que permiten ejecutar el proceso de integración y despliegue de manera automatizada.

Esta configuración contiene los ficheros necesarios de la herramienta de CI/CD seleccionada, así como las variables secretas requeridas para el despliegue en la infraestructura Cloud y la conexión con otros servicios.

Esto hace que el usuario no necesite modificar el código fuente en ningún momento.

### 5.4.2 Gestión de variables sensibles

El proyecto presta especial atención a la gestión de la información sensible necesaria para el despliegue de aplicaciones, como credenciales, claves de acceso o variables de entorno. Estos datos no se almacenan de forma explícita en el código fuente.

El asistente muestra al usuario como definir variables sensibles que deben configurarse como secretos en las herramientas de CI/CD.

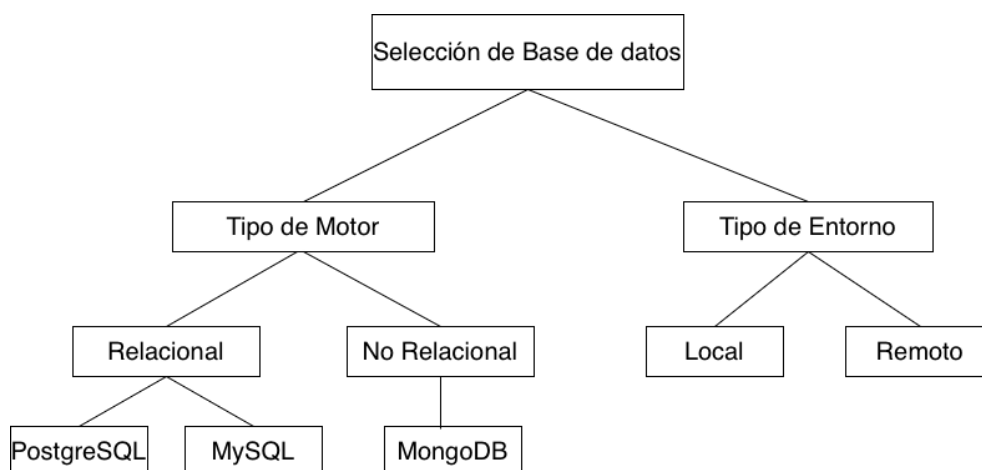
Este permite reducir riesgos de seguridad y se alinea con las buenas metodologías habituales en entornos DevOps y de despliegue en la nube.

## 5.5 Gestión de bases de datos

Los motores de bases de datos como PostgreSQL, MySQL o MongoDB permiten gestionar la persistencia de datos en aplicaciones modernas ([PostgreSQL Global Development Group, s. f.](#); [Oracle, s. f.](#); [MongoDB Inc., s. f.](#)).

Para ser más concretos, el asistente ofrece soporte para los motores PostgreSQL, MySQL y MongoDB.

Esta selección la realiza el usuario y queda asociada a la aplicación creada. Según el motor elegido, el sistema adapta la configuración generada y las variables necesarias para el despliegue, garantizando la coherencia entre el entorno y la aplicación.



*Ilustración 17 Diagrama de las bases de datos.*

## 5.6 Seguridad en las aplicaciones

La seguridad siempre es un aspecto muy importante tanto en el desarrollo de aplicaciones demo como en el proyecto asociado al TFG, ya que permite gestionar información sensible relacionada con configuraciones de despliegues, credenciales y variables de entorno. Por ese motivo, este proyecto inserta una serie de mecanismos de seguridad orientados a proteger el acceso a la aplicación y a los recursos gestionados por el despliegue.

El frontend incorpora un sistema de autenticación basado en Spring Security, que permite proteger el acceso al asistente de despliegues y garantizar que únicamente los usuarios registrados puedan utilizar la aplicación, incluyendo esta característica en los proyectos demo si usa esta tecnología. Para reforzar la seguridad del sistema, las contraseñas de los usuarios no se almacenan en texto plano en la base de datos. En su lugar, se utiliza un mecanismo de hash seguro proporcionado por Spring Security, concretamente mediante el algoritmo BCryptPasswordEncoder.

Para ello se ha configurado un sistema de registro y autenticación de usuarios que permite crear cuentas nuevas y acceder con sus credenciales validadas.

Además del control de acceso mediante autenticación, el sistema protege las diferentes funcionalidades y recursos de la aplicación para evitar accesos no autorizados. De este modo, operaciones como la creación, modificación, eliminación de las configuraciones únicamente puedan ser realizadas por el usuario propietario.

En las aplicaciones demo desarrolladas en Python/Flask también se incorporan mecanismos básicos de seguridad orientados a la autenticación y gestión segura de sesiones de usuario. Para ello, se utilizan librerías como Flask-Login, permitiendo implementar funcionalidades de inicio y cierre de sesión, gestión

de usuarios y almacenamiento seguro de contraseñas mediante funciones hash.

Asimismo, las demos utilizan variables de entorno para gestionar parámetros sensibles de configuración, como claves secretas o cadenas de conexión a bases de datos.

## 5.7 seguridad en el acceso a servicios de terceros

El sistema requiere la integración con diferentes servicios externos durante el proceso de generación y despliegue de aplicaciones. Para evitar exponer información sensible dentro del código fuente, las credenciales y parámetros de configuración se gestionan mediante variables seguras configuradas como secretos dentro de la plataforma CI/CD utilizada.

Estos mecanismos de configuración segura se emplean tanto en la infraestructura interna del proyecto como en las configuraciones de despliegue generadas automáticamente por el asistente. Durante el flujo de configuración, la aplicación proporciona al usuario instrucciones y manuales integrados en cada paso del asistente, indicando las variables y secretos necesarios para establecer correctamente la conexión con servicios externos como plataformas CI/CD, herramientas de análisis de calidad y servicios Cloud.

La Tabla 2 recoge los principales secretos utilizados durante el proceso de integración y despliegue automatizado.

| <i>Servicio</i> | <i>Variable</i>       | <i>Función</i>                               |
|-----------------|-----------------------|----------------------------------------------|
| AWS             | AWS_ACCESS_KEY_ID     | Identificador de acceso a AWS                |
| AWS             | AWS_SECRET_ACCESS_KEY | Clave secreta de autenticación AWS           |
| AWS             | AWS_REGION            | Región utilizada para desplegar servicios    |
| AWS             | AWS_ACCOUNT_ID        | Identificador de cuenta AWS                  |
| AWS             | ECR_REPOSITORY        | Repositorio de imágenes Docker en Amazon ECR |



|                      |                    |                                                 |
|----------------------|--------------------|-------------------------------------------------|
| <b>AWS EC2</b>       | EC2_HOST           | Dirección de la instancia EC2                   |
| <b>AWS EC2</b>       | EC2_USER           | Usuario SSH de la instancia                     |
| <b>AWS EC2</b>       | EC2_LLAVE_SSH      | Clave privada utilizada para acceso SSH         |
| <b>AWS EC2</b>       | EC2_KNOWN_HOSTS    | Verificación segura de hosts SSH                |
| <b>SonarCloud</b>    | SONAR_TOKEN        | Token de autenticación para análisis de calidad |
| <b>SonarCloud</b>    | SONAR_PROJECT_KEY  | Identificador del proyecto en SonarCloud        |
| <b>SonarCloud</b>    | SONAR_ORGANIZATION | Organización asociada al proyecto               |
| <b>SonarCloud</b>    | SONAR_HOST_URL     | URL del servicio SonarCloud                     |
| <b>Base de datos</b> | DB_USER            | Usuario de acceso a la base de datos            |
| <b>Base de datos</b> | DB_PASSWORD        | Contraseña de acceso a la base de datos         |
| <b>Base de datos</b> | DB_NAME            | Nombre de la base de datos                      |
| <b>Base de datos</b> | DB_PORT            | Puerto de conexión                              |
| <b>Base de datos</b> | DB_ENGINE          | Motor de base de datos utilizado                |

*Tabla 2: variables utilizadas para la integración con servicios externos*

En herramientas como GitHub Actions o GitLab CI/CD, los pipelines se integran directamente dentro del propio repositorio mediante archivos de configuración incluidos en el proyecto. Esto permite centralizar tanto el control de versiones como los procesos automatizados de integración y despliegue continuo.

Además, estas herramientas requieren mecanismos de autenticación seguros para permitir la interacción entre el entorno local de desarrollo, los repositorios remotos. Para ello, se utilizan credenciales seguras y tokens de acceso personal, evitando el uso directo de contraseñas y mejorando la seguridad durante las operaciones de acceso y modificación del repositorio.

En cambio, Jenkins utiliza una arquitectura desacoplada, donde la ejecución de los pipelines se realiza desde un servidor externo de automatización. En este caso, Jenkins obtiene el código fuente desde repositorios GitHub o GitLab y ejecuta las tareas definidas en el pipeline de forma independiente a la herramienta de control de versiones.

## 5.8 Manuales y tutoriales

El sistema ofrece distintos mecanismos de ayuda con el objetivo de facilitar el uso del asistente de despliegues y mejorar la comprensión del proceso por parte del usuario. Están orientados a ofrecer apoyo tanto durante la configuración inicial como a lo largo de los diferentes pasos del wizard de despliegue.

La decisión de insertar manuales y tutoriales responde al enfoque formativo del proyecto, permitiendo que los usuarios con menos experiencia puedan conseguir su despliegue de una forma guiada y progresiva.

### 5.8.1 Wizards guiados

La aplicación contiene tutoriales guiados que acompañan al usuario durante la interacción con la interfaz. Estos wizards explican de manera simple los distintos campos, indicando que valores deben introducirse y cuál es su función en el despliegue.

Este mecanismo de ayuda se activa desde la interfaz donde aparece el listado de las aplicaciones creadas y permite recorrer los diferentes pasos de forma secuencial. Con esta ayuda, se intenta que el usuario no consulte información externa.

### 5.8.2 Manuales de apoyo

Además de los tutoriales, en cada paso se habilita manuales de apoyo en formato PDF. Estos manuales proporcionan explicaciones más detalladas sobre las acciones que se deben llevar a cabo en cada fase del proceso.

Los manuales también se encuentran accesibles desde la interfaz del asistente, permitiendo su consulta en cualquier momento del paso sin interrumpir su flujo.

Estos manuales son más precisos ya que indica que hay que crear, como hay que crearlo y las variables que se van a obtener en cada paso.



Ilustración 18: Manuales para el usuario.

## 6. TRABAJOS RELACIONADOS

En este punto se presentan algunas herramientas, trabajos y soluciones relacionadas con el ámbito del despliegue de aplicaciones, la integración continua y la automatización en entornos Cloud. Su objetivo es contextualizar el desarrollo dentro del estado actual de las soluciones existentes y destacar las aportaciones que lo diferencian.

### 6.1 Proyectos y soluciones similares

En el ámbito profesional pueden existir diferentes herramientas orientadas a la integración y despliegue continuo de aplicaciones, como plataformas de CI/CD y servicios Cloud que ayudan a automatizar procesos de construcción, pruebas y despliegues. Estas soluciones suelen dirigirse a usuarios con conocimientos previos y necesitan una configuración manual más detallada.

Entre las herramientas más utilizadas se encuentran plataformas como GitHub Actions, Gitlab CI o Jenkins, que permiten definir pipelines de integración continua y despliegue automatizado mediante archivos de configuración ([GitHub, s. f.](#); [Jenkins, s. f.](#)).

Asimismo, los entornos Cloud han facilitado el despliegue de aplicaciones mediante servicios escalables y configurables bajo demanda. Plataformas como Amazon Web Services (AWS), permiten desplegar aplicaciones y gestionar infraestructuras sin necesidad de administrar directamente el hardware subyacente ([Mell & Grance, 2011](#); [Amazon Web Services, 2023](#)).

También existen herramientas específicas para la gestión de infraestructuras y despliegues automatizados que permiten definir entornos complejos mediante archivos de configuración. Aunque estas soluciones ofrecen una gran flexibilidad, su curva de aprendizaje puede resultar elevada para usuarios en proceso de formación.

En el ámbito académico, pueden encontrarse trabajos que abordan la automatización de despliegues o el uso de metodologías DevOps, generalmente centrados en herramientas concretas.

### 6.2 Comparación con otros trabajos

El proyecto desarrollado se centra en ofrecer un asistente de despliegue guiado, cuyo objetivo es ayudar al usuario a comprender el proceso completo de automatización desde un enfoque progresivo y estructurado.

Mientras otras herramientas requieren la configuración manual de pipelines y recursos de infraestructura, el sistema de este proyecto abstrae estos detalles y guía al usuario en todo el proceso, permitiendo reducir la complejidad del proceso.

Además, el soporte de diferentes herramientas CI/CD y motores de bases de datos permite adaptar el despliegue a diferentes escenarios.

Otra de las diferencias que podemos encontrar en nuestro sistema son las ayudas y manuales que podemos visualizar a lo largo de nuestro wizard.

### 6.3 Aportaciones del proyecto

Las principales aportaciones del proyecto desarrollado pueden definirse en los siguientes puntos:

- Un asistente de despliegue guiado que estructura el proceso de automatización en pasos claros y secuenciales.
- Integrar diferentes herramientas CI/CD dentro de un mismo sistema.
- Soportar múltiples motores de bases de datos, de tipo relacional o no relacional.
- Generación automática de recursos de configuración para el despliegue.
- Tener un enfoque educativo con tutoriales interactivos y manuales de apoyo.
- Aplicación práctica de conceptos relacionales con Cloud Computing, DevOps y automatización.

Este proyecto proporciona una solución orientada al aprendizaje y a la demostración de buenas prácticas en el ámbito del despliegue automatizado, diferenciándose de otras herramientas que están diseñadas para entornos puramente productivos.

## 7. USO DE INTELIGENCIA ARTIFICIAL EN EL DESARROLLO

Durante el desarrollo de este TFG se ha consultado herramientas de inteligencia artificial como apoyo en determinadas fases. Su uso se ha realizado de forma responsable y clara, siempre como complemento al trabajo del autor y no como sustitución del proceso de análisis o desarrollo.

El objetivo de su uso ha sido facilitar la comprensión de diferentes conceptos, manteniendo la supervisión y validación de las soluciones propuestas en todo momento.

### 7.1 Contexto del uso de IA

La utilización de la IA se ha producido en una forma formativa, como apoyo en alguna parte del desarrollo del proyecto. Se ha utilizado para resolver dudas concretas, revisar diseños o mejorar algunos fragmentos de código.

Esto ha permitido aprovechar las ventajas de la IA sin comprometer el aprendizaje y poder llevar a cabo lo aprendido a lo largo de esta titulación.

## 7.2 Ámbitos en los que se ha utilizado

La inteligencia artificial se ha utilizado principalmente en las siguientes fases:

- Apoyo en la comprensión de conceptos relacionados con DevOps, CI/CD y Cloud Computing.
- Obtener sugerencias para la estructura de algunos componentes del sistema
- Revisión y mejoras de fragmentos de código, siempre bajo la supervisión del autor.
- Ayudar con dudas técnicas puntuales durante el desarrollo.
- Diseñar alguna de las imágenes de la documentación.

Todas las propuestas generadas por la IA han sido analizadas, adaptadas y validadas antes de integrarlas en el proyecto.

## 7.3 Limitaciones y control del uso

Su uso se ha limitado a tareas de consulta y apoyo. Este control ha permitido mantener la coherencia del proyecto y demostrar los conocimientos aprendidos durante el grado de una forma práctica.

## 7.4 Responsabilidad del autor

El autor se hace cargo de la responsabilidad del resultado obtenido en el TFG presentado. Las decisiones de diseño, implementación y desarrollo han sido tomadas de forma razonada.

# 8. RESULTADOS Y VALIDACIÓN

En este apartado se exponen los resultados que se han obtenido tras realizar el proyecto, así como la validación funcional. Su objetivo es comprobar que la aplicación cumple con los requisitos planteados y que el asistente de despliegues permite realizar correctamente las configuraciones planteadas.

La validación se hace desde un enfoque práctico, evaluando los escenarios y las funcionalidades del sistema.

## 8.1 Resultados obtenidos

En este TFG se ha llevado a cabo una aplicación web funcional que actúa como asistente de despliegues para aplicaciones orientadas a entornos Cloud. Este sistema permite crear y gestionar aplicaciones donde se le guía al usuario a través de unos pasos estructurados de configuración y generar los recursos y variables necesarios para su despliegue automatizado.

El asistente funciona de una manera secuencial, validando cada uno de los pasos antes de permitir avanzar al siguiente. Esto hace que las configuraciones sean las correctas y reduce la posibilidad de fallos.

El sistema soporta diferentes herramientas CI/CD y motores de bases de datos, esto hace que se adapte cada entorno a lo que más necesita el usuario. El usuario dispone de manuales y tutoriales que hace que sea más fácil el uso de la aplicación.

Los resultados demuestran que el sistema cumple con los objetivos funcionales y técnicos definidos en el inicio del proyecto.

## 8.2 Validación funcional del sistema

La validación funcional se ha producido mediante la ejecución y comprobación de las principales funcionalidades de la aplicación. Se ha decidido realizar pruebas manuales centradas en los flujos de uso más relevantes.

Algunas de las validaciones realizadas son:

- Registro e inicio de sesión de usuarios.
- Creación y gestiones de diferentes aplicaciones.
- Ejecución completa del asistente de despliegues, comprobando el correcto avance entre los pasos.
- Generar archivos y recursos de configuración en función de las opciones elegidas por el usuario.
- Comprobar el acceso restringido a recursos según el usuario autenticado.
- Verificar el acceso a manuales y tutoriales que se encuentran dentro de la aplicación.

Después de realizar las pruebas anteriores, podemos confirmar que el sistema cumple de manera correcta los objetivos establecidos en este proyecto

## 8.3 Limitaciones actuales

A pesar de obtener los resultados previstos, el sistema tiene una serie de limitaciones derivadas del alcance definido para el proyecto. El asistente está enfocado a escenarios de despliegues controlados y no aborda configuraciones avanzadas propias de entornos de producción a gran escala.

El sistema no contempla aspectos como la alta disponibilidad, el balanceo de carga o la orquestación avanzada de contenedores, ya que estos requisitos quedan fuera del alcance del Proyecto.

Otra limitación es que la validación del sistema se centró en pruebas funcionales manuales, sin añadir pruebas automatizadas o métricas de rendimiento.

## 9. CONCLUSIONES Y LÍNEAS DE TRABAJO FUTURAS

En este apartado se presentan las conclusiones obtenidas tras el desarrollo del proyecto, analizando los resultados alcanzados desde el punto de vista técnico y académico. Además, se identifican posibles líneas de trabajo futuro que permitan ampliar o mejorar el sistema desarrollado.

### 9.1 Conclusiones técnicas

Desde la parte técnica, el proyecto ha permitido diseñar y desarrollar una aplicación web funcional orientada a la automatización del despliegue de aplicación en entornos Cloud. Ofrece un asistente de despliegues guiado que estructura el proceso en pasos claros y comprobados.

La integración tanto de diferentes herramientas de CI/CD, como de múltiples motores de bases de datos, demuestra la flexibilidad de la solución y su capacidad para adaptarse a diferentes escenarios. La generación automática de configuraciones hace que se reduzcan los errores que suelen ser habituales y a simplificar procesos que pueden ser complejos para usuarios con menor experiencia. El uso de Docker y de entornos de despliegue es consecuente con el enfoque cultural de CI/CD.

### 9.2 Conclusiones académicas

Desde el punto de vista académico, este proyecto ha permitido aplicar de forma práctica los conocimientos aprendidos a lo largo del grado, utilizando conceptos relacionados en el desarrollo del software, Cloud Computing, DevOps e integración continua.

Este TFG ha permitido consolidar competencias como el análisis de requisitos, toma de decisiones técnicas, diseño de soluciones y documentación de un sistema complejo. Esto ha ayudado a entender el ciclo de vida de un software desde su concepción hasta su despliegue automatizado.

Este proyecto ha facilitado el aprendizaje progresivo de tecnologías y herramientas que se utilizan a menudo en entornos profesionales.

### 9.3 Certificaciones técnicas relacionadas con el proyecto

Durante el desarrollo del TFG he obtenido diferentes certificaciones con tecnologías Cloud y arquitecturas de despliegue utilizadas en el proyecto. Estas certificaciones han permitido reforzar los conocimientos necesarios para comprender y aplicar conceptos asociados a infraestructuras Cloud, automatización de despliegues, seguridad y diseño de arquitecturas escalables sobre Amazon Web Services.

## AWS Certified Cloud Practitioner

La certificación AWS Certified Cloud Practitioner (Ilustración 19) valida conocimientos fundamentales sobre computación en la nube y sobre los principales servicios ofrecidos por AWS. Entre los contenidos abordados destacan los conceptos generales de Cloud Computing, modelos de responsabilidad compartida, principios básicos de seguridad, gestión de identidades y acceso (IAM), almacenamiento, redes, monitorización y modelos de costes dentro de la plataforma AWS.

La obtención de esta certificación ha permitido consolidar los conocimientos básicos necesarios para comprender el funcionamiento de los servicios Cloud utilizados en el proyecto, especialmente aquellos relacionados con el despliegue y gestión de aplicaciones en infraestructura AWS.



*Ilustración 19: certificación AWS Certified Cloud Practitioner*

## AWS Certified Solutions Architect- Associate

La certificación AWS Certified Solutions Architect- Associate (Ilustración 20) está orientada al diseño de arquitecturas distribuidas y escalables dentro del ecosistema AWS. Esta certificación se centra en el diseño de soluciones resilientes, seguras, eficientes y de alta disponibilidad utilizando servicios Cloud.

Entre los conocimientos trabajados destacan el diseño de arquitecturas basadas en servicios como Amazon EC2, Amazon ECR, redes VPC, balanceadores de carga, almacenamiento, mecanismos de seguridad, integración entre servicios y estrategias de despliegue automatizado.

Obtener esta certificación durante el desarrollo del TFG ha resultado especialmente útil para comprender y aplicar decisiones técnicas relacionadas con la infraestructura Cloud utilizada en el proyecto, así como para diseñar procesos de despliegue automatizado y arquitecturas basadas en contenedores Docker dentro de AWS.





*Ilustración 20: certificación AWS Certified Solutions Architect- Associate*

## 9.4 Trabajo futuro

Se puede identificar futuras ideas de trabajo permitiendo ampliar sus funcionalidades y mejorar su alcance. Algunas de ellas pueden ser incorporar nuevas herramientas de CI/CD y ampliar el soporte a otros servicios Cloud.

Se podría estudiar la posibilidad de integrar nuevos mecanismos más avanzados de despliegue, como la orquestación de contenedores, balanceo de carga o una alta disponibilidad, orientada a entornos de producción a gran escala.

Otras funciones que se pueden insertar en un futuro son las pruebas automatizadas, métricas de rendimiento o sistemas de monitorización.

Estas ideas pueden permitir tener un software más avanzado y que de una solución más completa, manteniendo su parte educativa y su utilidad como herramienta de aprendizaje.

## 10. BIBLIOGRAFÍA

Amazon Web Services. (s. f.). AWS documentation.

<https://docs.aws.amazon.com>

Spring. (s. f.). Spring Boot reference documentation.

<https://docs.spring.io/spring-boot/tutorial/index.html>

Docker. (s. f.). Docker documentation.

<https://docs.docker.com/manuals/>

GitHub. (s. f.). GitHub Actions documentation.

<https://docs.github.com/actions>

GitLab. (s. f.). GitLab CI/CD documentation.

<https://docs.gitlab.com/ee/ci/>

Jenkins. (s. f.). Jenkins user documentation.

<https://www.jenkins.io/doc/>

SonarSource. (s. f.). SonarCloud documentation.

<https://docs.sonarcloud.io/>

Flyway. (s. f.). Flyway documentation.

<https://documentation.red-gate.com/fd>

PostgreSQL Global Development Group. (s. f.).

PostgreSQL documentation. <https://www.postgresql.org/docs/>

Oracle. (s. f.). MySQL documentation.

<https://dev.mysql.com/doc/>

MongoDB Inc. (s. f.). MongoDB documentation.

<https://www.mongodb.com/docs/>

Fowler, M. (2006). Continuous integration.

<https://martinfowler.com/articles/continuousIntegration.html>

Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations.

<https://itrevolution.com/the-devops-handbook/>

Mell, P., & Grance, T. (2011). The NIST Definition of Cloud Computing.

<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>

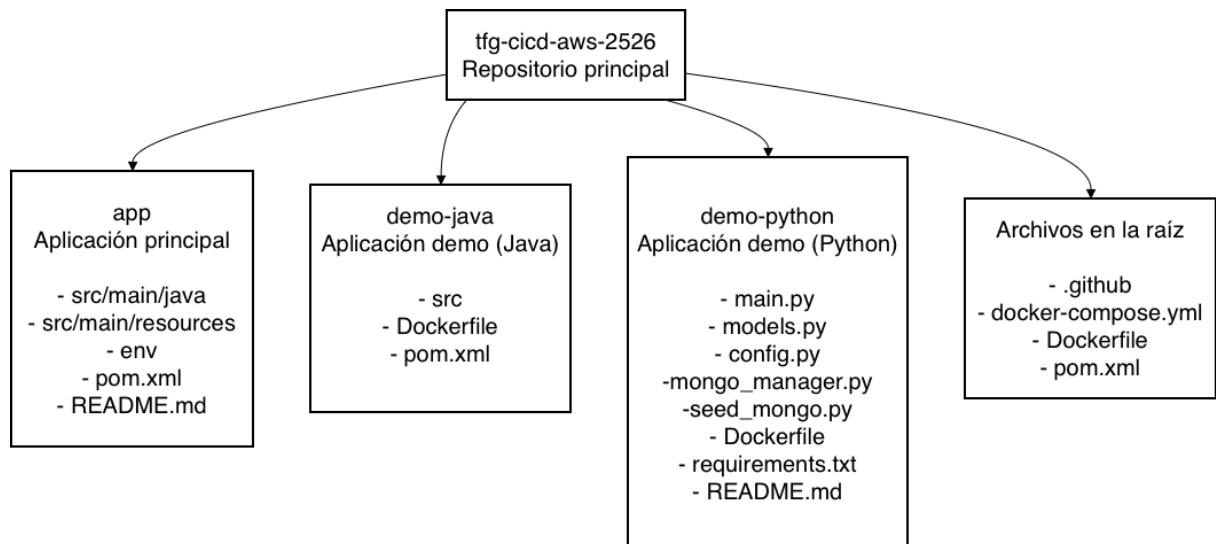
Amazon Web Services. (2023). AWS Overview.

<https://docs.aws.amazon.com/whitepapers/latest/aws-overview/aws-overview.pdf>

IONOS. (s. f.). ¿Qué es DevOps?. IONOS Digital Guide.

## ANEXOS

### Anexos A. Estructura del proyecto



*Ilustración 21: Vista simplificada de la estructura del proyecto.*

En la Ilustración 21 se omiten los directorios generados automáticamente y archivos no relevantes para facilitar la visión de la organización general del proyecto.

La aplicación principal sigue una estructura típica de aplicaciones web basadas en frameworks Java, separando la lógica de negocio, gestión de datos y la presentación.

### Anexo B. Licencias

#### B.1 Licencia del proyecto desarrollado

El proyecto desarrollado en este TFG se publica bajo licencia MIT, se permite el uso, modificación y distribución, siempre que se mantenga el copyright correspondiente.

Se elige esta licencia por su carácter permisivo y a su amplia utilización en proyectos de código abierto.

#### B.2 Uso de proyectos de terceros

Durante el proceso de desarrollo, se ha utilizado como base una aplicación de demostración desarrollada en Python, la cual se ha seleccionado desde GitHub con licencia MIT, siendo un repositorio abierto.

Comparto el repositorio utilizado:

[https://github.com/ayushi-gajendra/Flask\\_Authentication\\_App](https://github.com/ayushi-gajendra/Flask_Authentication_App)

Este repositorio se ha utilizado como base, para adaptarlo y modificarlo para la demo del código Python. En ella se ha añadido un CRUD en el diseño y la adaptación para conseguir el despliegue deseado con los diferentes entornos y motores de bases de datos.

### B.3 Otras herramientas y dependencias

En este proyecto se utilizan diferentes herramientas y librerías de terceros, como frameworks, motores de bases de datos y herramientas de automatización, todas ellas sujetas a licencias de software libre o de código abierto.